OS LAB09 SUBMISSION

Name: Yash Raj
Roll Number: 24k-0737

Task03:

```c
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

int data = 0;   // shared variable (simulating IPC)

void handler(int sig) {
    printf("Signal received! Data = %d\n", data);
}

int main() {

    signal(SIGUSR1, handler);

    pid_t pid = fork();

    if (pid == 0) {
        while (1) {
            pause(); // wait for signal
        }
    }
    else {
        for (int i = 1; i <= 5; i++) {
            sleep(1);

            data = i * 10;  // write data (IPC part)

            printf("Parent sending data = %d\n", data);

            kill(pid, SIGUSR1);  // signal child
        }
    }

    return 0;
}
```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc s6.c -o D
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./D
Parent sending data = 10
Signal received! Data = 0
Parent sending data = 20
Signal received! Data = 0
Parent sending data = 30
Signal received! Data = 0
Parent sending data = 40
Signal received! Data = 0
Parent sending data = 50
Signal received! Data = 0
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$
```

Task02:

```c
// Code Workout # 3 (Threads + Signals)

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>

pthread_t threads[3];

// Signal handler
void handler(int sig) {
    printf("Signal %d handled by thread %lu\n", sig, pthread_self());
}

// Thread function
void* worker(void* arg) {
    int id = *(int*)arg;

    // Uncomment this to register handler inside thread
    // signal(SIGUSR1, handler);

    printf("Thread %d started (TID: %lu)\n", id, pthread_self());

    while (1) {
        sleep(1);
    }

    return NULL;
}

```

```c
int main() {
    int ids[3] = {1, 2, 3};

    // Register handler in main thread
    signal(SIGUSR1, handler);

    // Create threads
    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }

    sleep(2);

    // Send signal to PROCESS
    printf("\nSending signal to PROCESS\n");
    kill(getpid(), SIGUSR1);

    sleep(2);

    // Send signal to specific THREAD (3rd thread)
    printf("\nSending signal to THREAD 3\n");
    pthread_kill(threads[2], SIGUSR1);

    sleep(2);

    return 0;
}
```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc s6.c -o D
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./D
Thread 1 started (TID: 126864741889728)
Thread 2 started (TID: 126864733497024)
Thread 3 started (TID: 126864725104320)

Sending signal to PROCESS
Signal 10 handled by thread 126864744511296

Sending signal to THREAD 3
Signal 10 handled by thread 126864725104320
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$
```

Task04:

```c

// Code Workout # 2 (SIGCHLD Handling – Zombie Removal)

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

// Handler for child termination
void proc_exit(int sig) {
    int status;
    pid_t pid;

    // Handle all terminated children
    while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
        printf("Child %d terminated\n", pid);
    }
}

int main() {
    signal(SIGCHLD, proc_exit);

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();

        if (pid == 0) {
            printf("Child %d started\n", getpid());
            sleep(2);
            printf("Child %d exiting\n", getpid());
            exit(0);
        }
    }

    // Parent sleeps → children become zombies briefly
    sleep(10);

    printf("Parent exiting\n");
    return 0;
}

```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc s6.c -o D
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./D
Child 705 started
Child 706 started
Child 707 started
Child 705 exiting
Child 706 exiting
Child 707 exiting
Child 705 terminated
Child 706 terminated
Child 707 terminated
Parent exiting
```

Task 01:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <pthread.h>

// Signal handler
void handler(int sig) {
    printf("Signal %d received in thread (TID: %lu)\n", sig, pthread_self());
}

// Thread function
void* thread_func(void* arg) {
    signal(SIGUSR1, handler);
    printf("Thread started (TID: %lu)\n", pthread_self());
    while (1) {

        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t tid;

    signal(SIGUSR1, handler);

    // Create thread
    if (pthread_create(&tid, NULL, thread_func, NULL) != 0) {
        perror("Thread creation failed");
        exit(1);
    }

    sleep(2);

    printf("\nSending SIGUSR1 to PROCESS using kill()\n");
    kill(getpid(), SIGUSR1);   // goes to any thread

    sleep(2);

    printf("\nSending SIGUSR1 to SPECIFIC THREAD using pthread_kill()\n");
    pthread_kill(tid, SIGUSR1);  // goes to that thread

    sleep(2);


    pthread_cancel(tid);
    pthread_join(tid, NULL);

    return 0;
}
```

Output:

```
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ gcc s6.c -o D
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$ ./D
Thread started (TID: 135470776317632)

Sending SIGUSR1 to PROCESS using kill()
Signal 10 received in thread (TID: 135470779250496)

Sending SIGUSR1 to SPECIFIC THREAD using pthread_kill()
Signal 10 received in thread (TID: 135470776317632)
yashraj@DESKTOP-A4HGUGP:/mnt/c/Users/yashm/OslabFinal$
```